pfi

Python for Fixed Income

COURSE 2 · BOND MATH

Time Value of Money

WEEK 1 · DAY 1

Created by **Jeff Lenamon**

This course is educational content created in a personal capacity. Nothing here is investment advice or a recommendation to buy or sell any security. All portfolio data is synthetic.

Course 2 opens: the desk lead's note

Course 1 ended with a project that tested a file against its cover note before trusting it. Course 2 turns that habit on the bond math itself. Over three weeks you write the desk's functions in one module of your own, test every one against Excel, and keep each day's work with Git.

The Desk Lead's Note

From today the desk computes its own prices, in one module you write, with Excel as the answer key. Four new bonds settle today, 30 September 2026. Start at the base: what 100 grows to at Quorvane's yield, and what each bond's 100 at maturity is worth today. Show me, in dollars, how much the compounding convention moves that. Put both functions in your own file, docstring first. Prove they match Excel on every row of the answer key. Save a version you can go back to, and take it home.

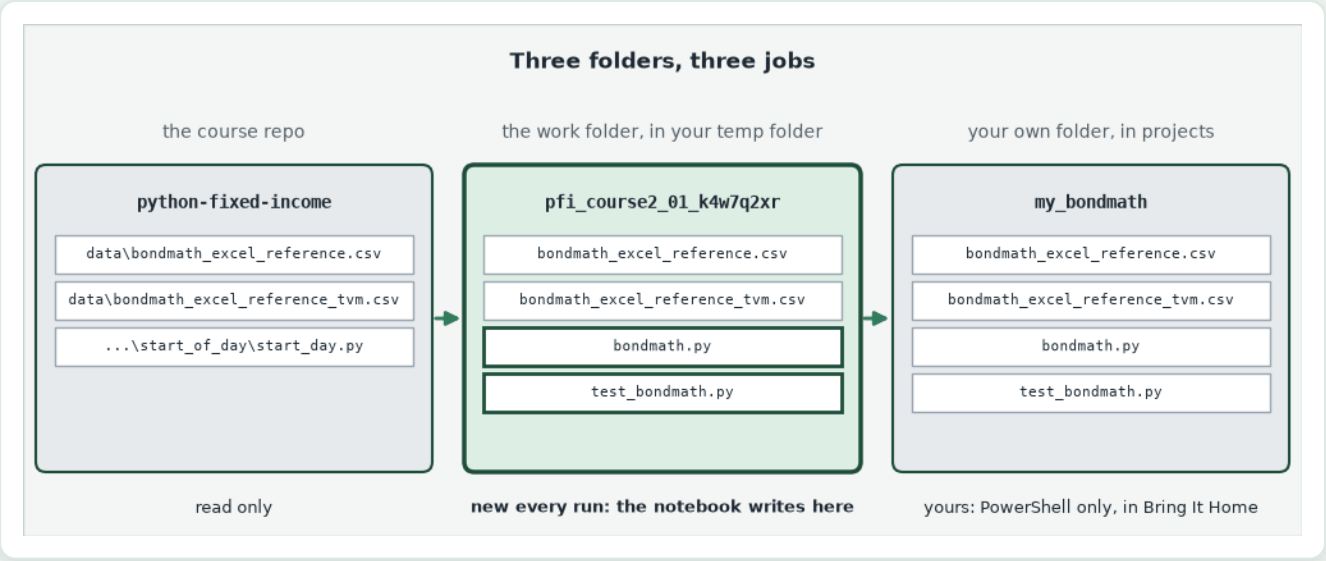
Two units hold in every function

Unit	Rule	Example
Rates in	percent a year	5.4 is 5.40 percent
Money out	per 100 of face	76.6118 for every 100

Excel takes decimals, so every Excel formula here divides by 100 in plain sight: "5.4/100". The issuers are invented; every number describes the data.

Excel has the answers; today you write the code that matches them.

1.1 The work folder, and one row of Excel's answers



Only the folder's name prints: its full path would show your user name. "work" holds that path; every later step uses it.

```
work = start_day(1)
Work folder pfi_course2_01_k4w7q2xr: 2 data files, no module yet
```

The work folder holds Excel's answers; everything the notebook writes lands beside them.

One Row of the Answer Key

```
tvm = pd.read_csv('bondmath_excel_reference_tvm.csv', index_col='case_id')
print(len(tvm), 'rows. Row tvm_fv_09:')
print(tvm.loc['tvm_fv_09', 'excel_formula'], 'returned', tvm.loc['tvm_fv_09', 'excel_value'])
153 rows. Row tvm_fv_09:
=FV(5.4/100/2,2*5,0,-100) returned 130.52822611608937
```

Today's job: reach that number in code, then prove the code on every row like it.

1.2 Future value: interest on interest

At 5.40 percent a year, compounded twice a year, 100 earns half the annual rate each half-year, and from then on that interest earns interest too. Future value is what the 100 has grown to at the end.

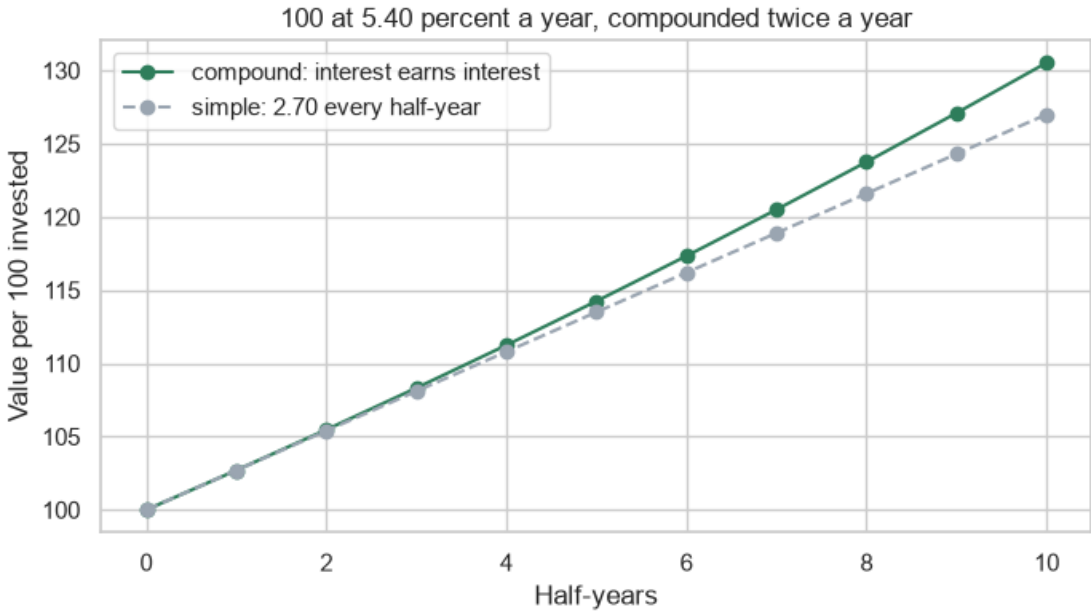
One Half-Year

```
period_rate = 5.4 / 100 / 2
Half-year rate: 0.027. 100 after one half-year: 102.70
```

Ten Half-Years

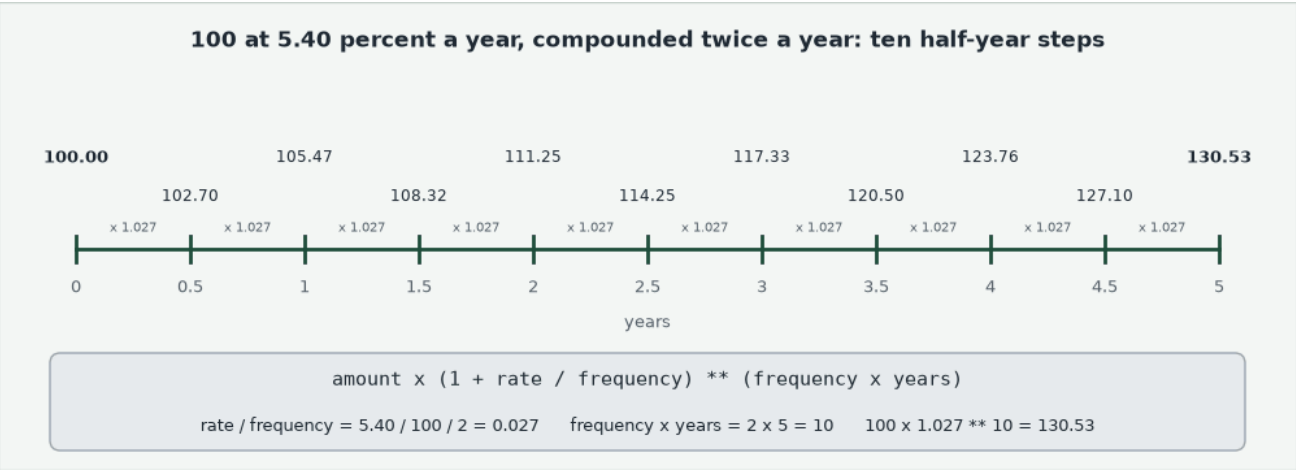
```
value = 100
rows = []
for half_year in range(1, 11):
    value = value * (1 + period_rate)
    rows.append({'half_year': half_year, 'value': round(value, 4)})
growth = pd.DataFrame(rows)
growth
```

Each step adds more than the one before: 2.70 in the first half-year, 3.43 in the last.



130.53 against 127.00: the gap, which widens every step, is interest earned on interest.

The ten steps in one formula




```
growth_factor = (1 + period_rate) ** (2 * 5)
Growth factor: 1.3053. Future value: 130.5282
```

Excel's 130.52822611608937, reached in one line.

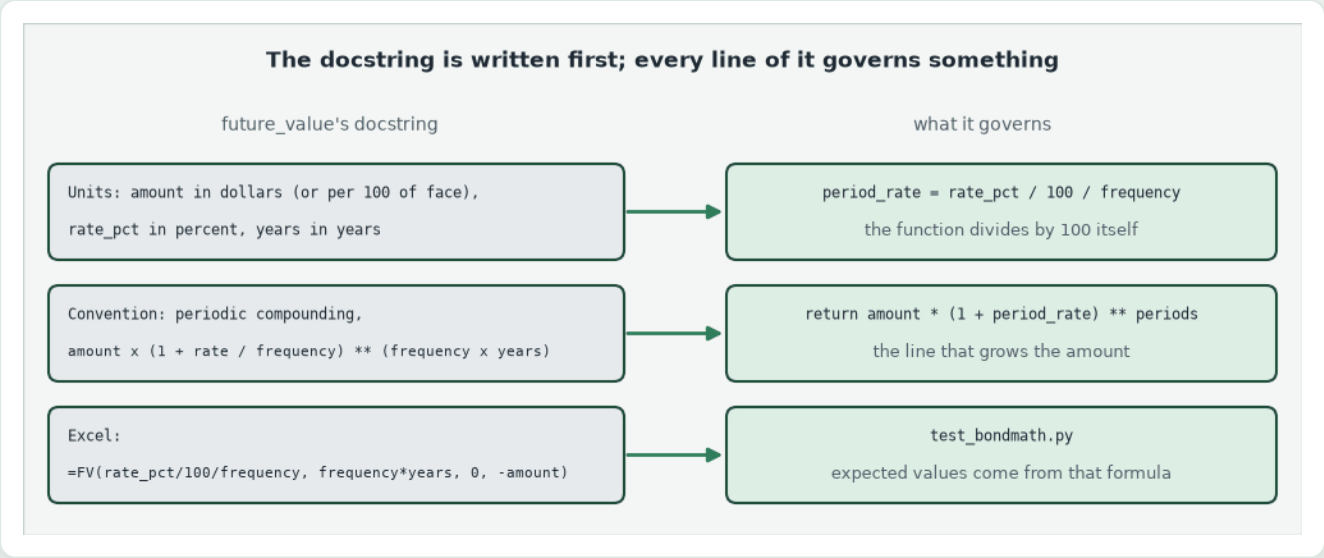
One formula grows 100 to Excel's 130.5282: the loop, written once.

Excel side by side

Job	Python	Excel	Result
Growth column	`the loop`	100 in A1, =A1*(1+5.4/100/2) in A2, filled to A11	130.5282
Simple column	`simple`	100 in D1, =D1+100*5.4/100/2 in D2, filled to D11	127.00
Growth factor	`(1 + period_rate) ** (2 * 5)`	=(1+5.4/100/2)^(2*5)	1.3053
Future value	`100 * growth_factor`	=FV(5.4/100/2,2*5,0,-100)	130.5282

"FV" takes the rate per period, the number of periods, a payment each period (0 here), and the amount entered as "-100"; it returns the grown amount as a positive number.

1.3 future_value in your own module, docstring first



The module's docstring sets the units every later function keeps. The function refuses a frequency other than 1, 2 or 4, then takes 1.2's two steps.

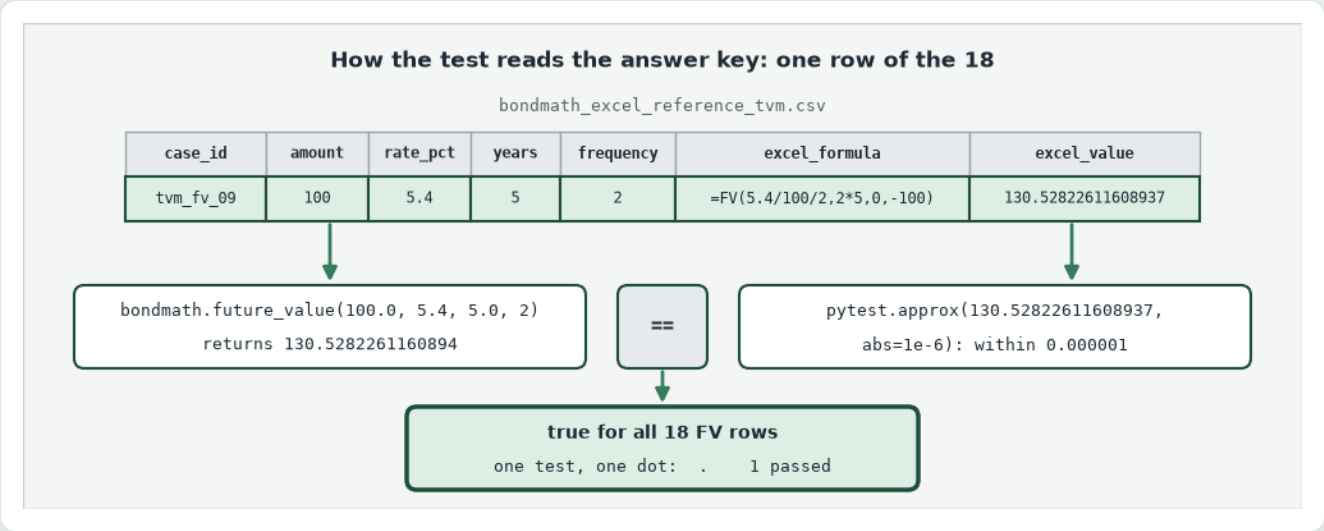
```
import bondmath
print(bondmath.future_value(100, 5.4, 5))
130.5282261160894
```

Notice "5.4", not "0.054": percent in, as the docstring promises. The frequency was left out, so its default, 2, applied.

The function lives in a file now, and its docstring says what it promises.

```
%%writefile bondmath.py
def future_value(amount: float, rate_pct: float, years: float, frequency: int = 2) -> float:
    if frequency not in (1, 2, 4):
        raise ValueError(f"frequency must be 1, 2, or 4, not {frequency}")
    period_rate = rate_pct / 100 / frequency
    periods = frequency * years
    return amount * (1 + period_rate) ** periods
```

1.4 Excel as the answer key: your first pytest run




```
def test_future_value_matches_excel_fv():
    for row in tvm_rows("FV"):
        amount, rate_pct = float(row["amount"]), float(row["rate_pct"])
        years, frequency = float(row["years"]), int(row["frequency"])
        result = bondmath.future_value(amount, rate_pct, years, frequency)
        assert result == pytest.approx(float(row["excel_value"]), abs=1e-6), row["case_id"]
```

"subprocess.run" runs a program and waits: here this notebook's own Python ("sys.executable") runs pytest's short report ("-m pytest -q") in the work folder ("cwd=work"), keeping what it prints as text for the cell to print. pytest starts fresh, so it reads both files as they are on disk now.

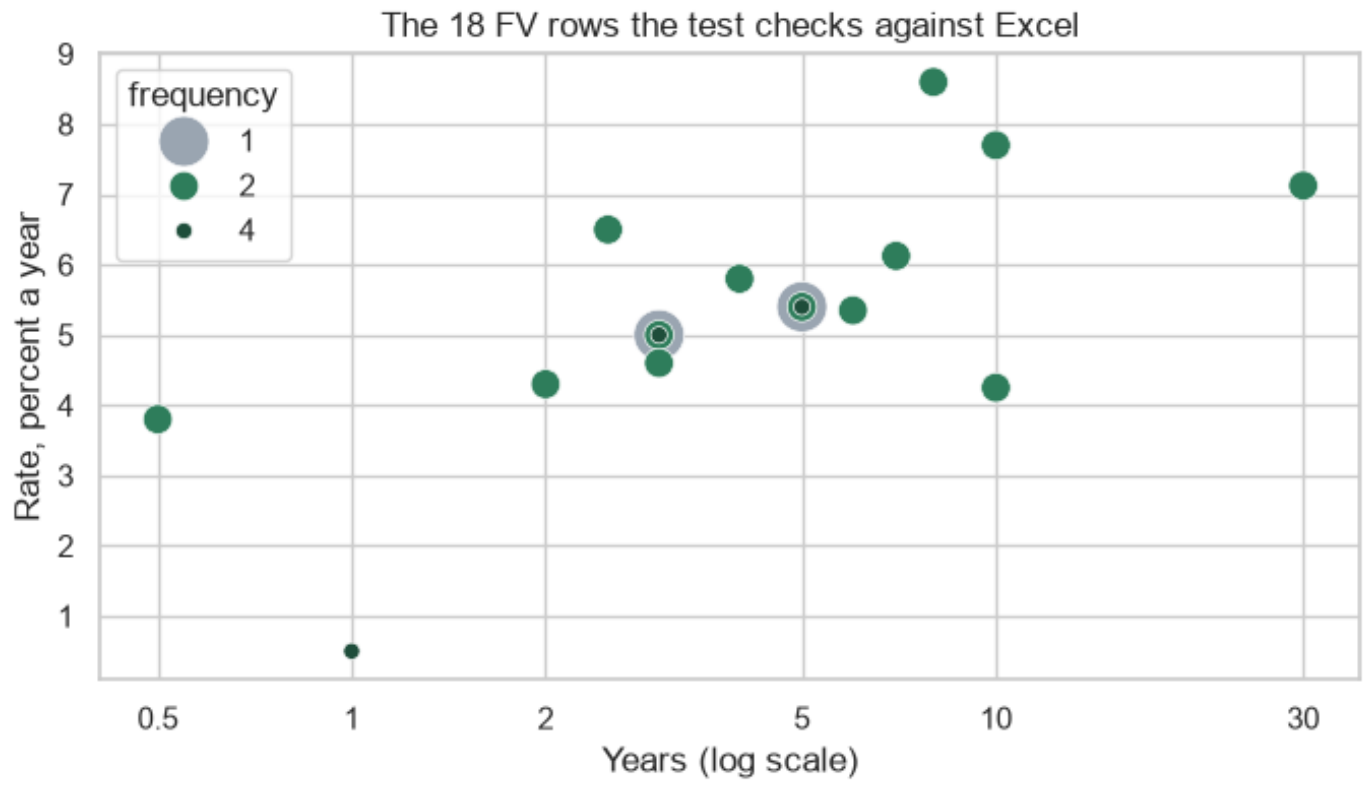
```
.   [100%]
1 passed in 0.50s
```

One dot, "1 passed": one test, every FV row of the answer key within 0.000001.

Excel side by side

A check column does the same job for one row: Python's 130.5282261160894 in A2, "=FV(5.4/100/2,2*5,0,-100)" in B2, and "=ABS(A2-B2)<0.000001" in C2 shows TRUE.

Which rows the one test read



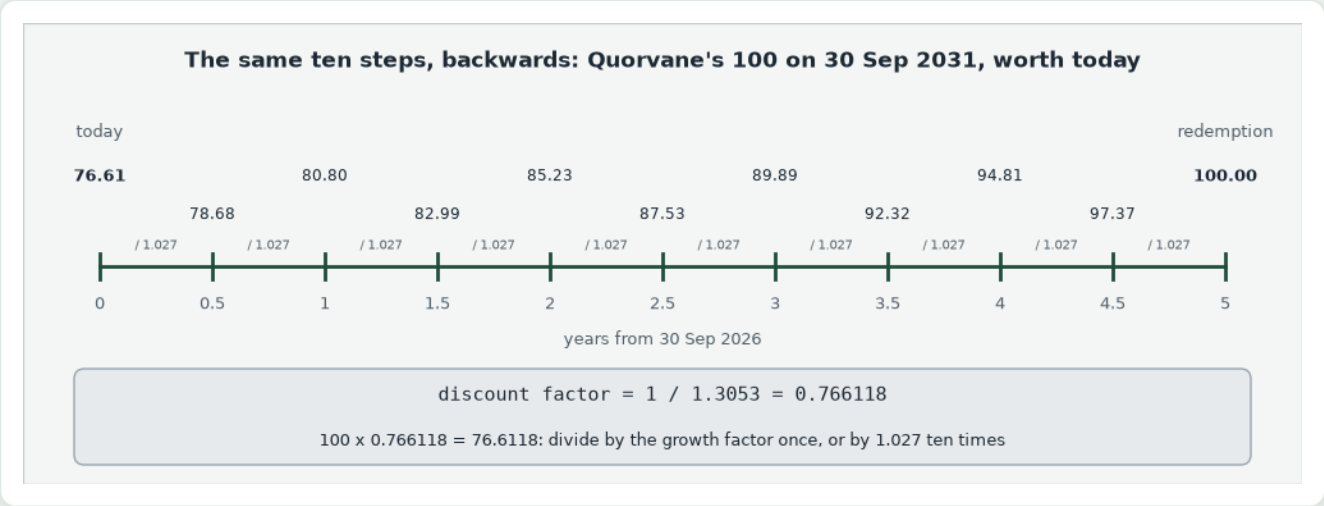
Which rows did that one test read?

```
fv_rows = tvm[tvm['function'] == 'FV']
18 FV rows: 0.5 to 30.0 years, 0.5 to 8.6 percent
Rows by frequency: {1: 2, 2: 13, 4: 3}
```

Short and long, low rate and high, all three frequencies: at 3 and 5 years the dots nest, Quorvane's among them. A test only covers the cases it reads.

"1 passed": your function agrees with Excel on all 18 FV rows.

1.5 Present value: the same steps, backwards



Quorvane repays 100 on 30 September 2031, its redemption. Its value today is the same ten steps run backwards. 1 divided by the growth factor is the discount factor of Course 1's Introduction, Part 1.

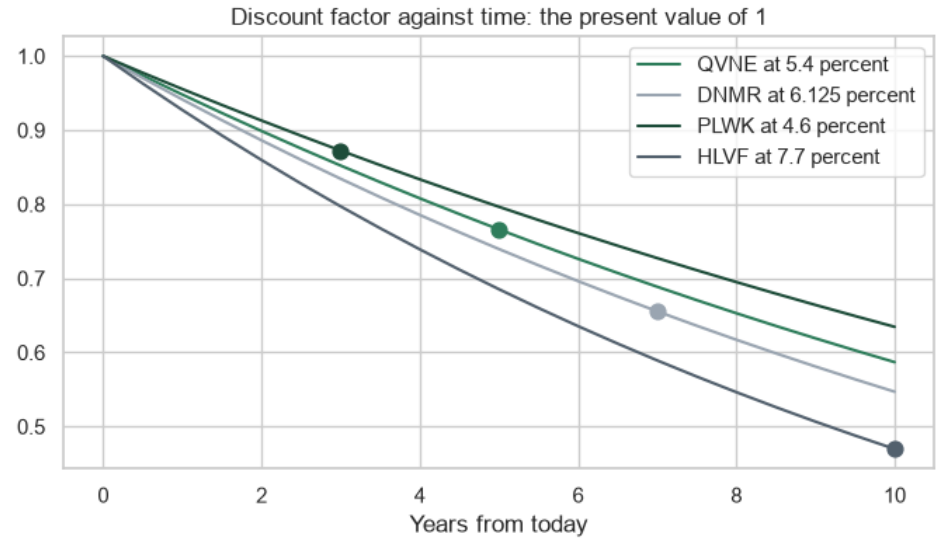
ticker	years	yield_pct	pv_of_100
QVNE	5	5.400	76.6118
DNMR	7	6.125	65.5527
PLWK	3	4.600	87.2461
HLVF	10	7.700	46.9753

A higher rate and a longer wait both push a present value down.

```
%%writefile -a bondmath.py
def present_value(amount: float, rate_pct: float, years: float, frequency: int = 2) -> float:
    if frequency not in (1, 2, 4):
        raise ValueError(f"frequency must be 1, 2, or 4, not {frequency}")
    period_rate = rate_pct / 100 / frequency
    periods = frequency * years
    return amount / (1 + period_rate) ** periods
```

The docstring settles the sign before anyone is surprised by it: positive here, negative from Excel's "PV".

Discount factors, Excel's PV, and three tests



Every line starts at 1 today and falls with the wait; the higher the yield, the faster. The dots are the table's numbers divided by 100.

"PV" returns money received later as money paid now: negative. "present_value" returns it positive, so its test compares with minus Excel's value.

The second test needs no Excel: grow 100 for 3 years at 5 percent, discount it back, and 100 must return at every frequency.

```
...   [100%]
3 passed in 0.50s
```

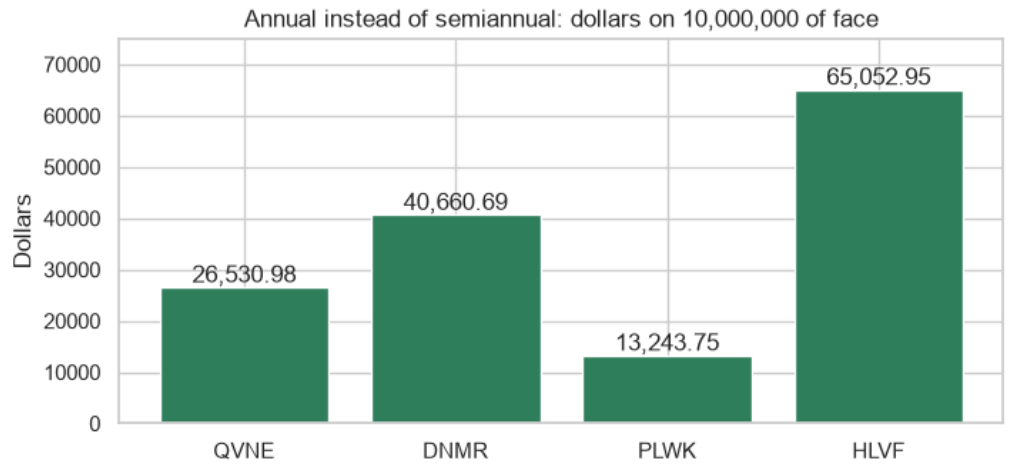
Discounting is growth run backwards, and "3 passed" covers both directions.

Excel side by side

Job	Python	Excel	Result
Discount factor	`bondmath.present_value(1, 5.4, 5)`	=1/(1+5.4/100/2)^(2*5) or =-PV(5.4/100/2,2*5,0,1)	0.766118
Present value, written out	`bondmath.present_value(100, 5.4, 5)`	=100/(1+5.4/100/2)^(2*5)	76.6118
Excel's PV		=PV(5.4/100/2,2*5,0,100)	-76.6118
PV, sign turned		=-PV(5.4/100/2,2*5,0,100)	76.6118

1.6 One rate, three frequencies

A rate of 5.40 percent a year can be applied once, twice, or four times a year. The course's bonds compound twice; a yield is not complete without its frequency.



Halvering, the longest at the highest yield, moves most: 65,052.95 dollars. That is a convention error: the code runs, the number looks reasonable, and it is wrong.

```
bondmath.future_value(100, 5.4, 5, frequency=12)
ValueError: frequency must be 1, 2, or 4, not 12
```

It refuses rather than guessing: a monthly number would look just as reasonable.

A yield is not complete without its frequency: the wrong one moves Halvering by 65,052.95 dollars, and nothing warns you.

Excel side by side. The frequency sits twice in each formula: in the rate and in the periods.

Job	Python	Excel	Result
Annual, grown	`bondmath.future_value(100, 5.4, 5, 1)`	=FV(5.4/100/1,1*5,0,-100)	130.0778
Annual, today	`bondmath.present_value(100, 5.4, 5, 1)`	=PV(5.4/100/1,1*5,0,100)	-76.8771
Quarterly, grown	`bondmath.future_value(100, 5.4, 5, 4)`	=FV(5.4/100/4,4*5,0,-100)	130.7600
Quarterly, today	`bondmath.present_value(100, 5.4, 5, 4)`	=PV(5.4/100/4,4*5,0,100)	-76.4760

1.7 A test that fails, and which side is wrong

A colleague writes a quick test of Quorvane's present value against Excel's "PV", exactly as Excel returns it.

```
%%writefile test_excel_sign.py
import pandas as pd
import pytest
import bondmath
def test_quorvane_against_excel_pv():
    excel = pd.read_csv('bondmath_excel_reference_tvm.csv', index_col='case_id')
    expected = excel.loc['tvm_pv_09', 'excel_value']
    assert bondmath.present_value(100, 5.4, 5) == pytest.approx(expected, abs=1e-6)
```

```
F   [100%]
>   assert bondmath.present_value(100, 5.4, 5) == pytest.approx(expected, abs=1e-6)
E   assert 76.61178196895271 == -76.61178196895273 ± 1.0e-06
E   Obtained: 76.61178196895271
E   Expected: -76.61178196895273 ± 1.0e-06
1 failed in 0.50s
```

A file name after the command runs only that file.

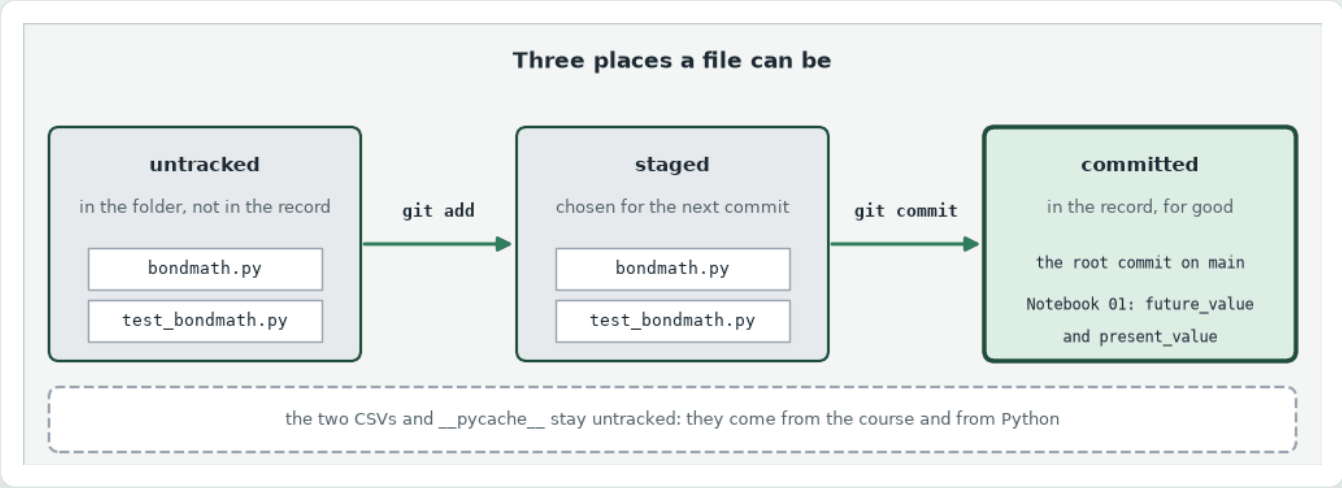
"F" instead of a dot, and "1 failed". Read it from the bottom, as in Course 1's Debugging notebook: the last line counts the failures, the one above names the test. The ">" line is the "assert"; the "E" line gives the two numbers, equal apart from the sign. The function is right and the test is wrong: "test_bondmath.py" already compares with minus Excel's value.

```
os.remove('test_excel_sign.py')
print(sorted(os.listdir()))
['__pycache__', 'bondmath.py', 'bondmath_excel_reference.csv', 'bondmath_excel_reference_tvm.csv', 'test_bondmath.py']
```

"__pycache__" is where Python keeps compiled copies of the modules it imports; leave it alone.

A failing test says two things disagree; deciding which is wrong is your job.

1.8 Save the day with Git: the first commit




```
!git -C "{work}" add bondmath.py test_bondmath.py
!git -C "{work}" status --short
A  bondmath.py
A  test_bondmath.py
?? __pycache__/
?? bondmath_excel_reference.csv
?? bondmath_excel_reference_tvm.csv
!git -C "{work}" commit -m "Notebook 01: future_value and present_value"
[main (root-commit) d029474] Notebook 01: future_value and present_value
 2 files changed, 105 insertions(+)
 create mode 100644 bondmath.py
 create mode 100644 test_bondmath.py
```

Git keeps every version of the files in a folder. The folder is a repository, and each saved version a commit, with a message saying what changed.

Excel side by side

Save As with a date keeps whole copies; a commit keeps each version with a message and the exact lines that changed.

Installing Git. Colab has it. On Windows, install Git for Windows from git-scm.com/downloads with the default choices, as your firm's policy allows, then restart VS Code.

Git sees every file in the folder: "??" marks one it keeps no record of, an untracked file. "A" marks a file staged for the next commit; the "??" files stay out. "root-commit" because it is the first; then the commit hash, the commit's name, and the message.

Today's two files are in the record, and every later change can be read line by line.

Bring it home: your own my_bondmath

Your own copy lives in "my_bondmath", beside the course repo; no notebook touches it. In a VS Code PowerShell terminal with the course's ".venv" active, run these lines with your own work folder, name and email:

```
$work = "$env:TEMP\pfi_course2_01_k4w7q2xr"
New-Item -ItemType Directory -Force "$HOME\projects\my_bondmath" | Out-Null
Set-Location "$HOME\projects\my_bondmath"
Copy-Item "$work\bondmath.py", "$work\test_bondmath.py" .
Copy-Item "$work\bondmath_excel_reference.csv", "$work\bondmath_excel_reference_tvm.csv" .
python -m pytest -q
git init -q -b main
git config user.name "Your Name"
git config user.email "you@example.com"
git add bondmath.py test_bondmath.py
git commit -m "Notebook 01: future_value and present_value"
```

```
...   [100%]
3 passed in 0.50s
```

Git then reports the root commit, "Notebook 01: future_value and present_value", and its 2 files. In Colab the work folder is under "/tmp": open it in the file browser on the left and download the two ".py" files. If Git is blocked on your laptop, read the output above. The work folder can stay: a restart makes a new one.

Your own "my_bondmath" holds the module, its tests, and their first commit.

Predict first: write down your answer, then run the cell

Predict 1

The first five years added 30.53 to the 100. Five more years at the same rate: what is it worth after ten? Write down a number.

```
ten_years = 100 * (1 + period_rate) ** (2 * 10)
second_five = ten_years - 100 * growth_factor
print(f'After ten years: {ten_years:.2f}. The second five years added {second_five:.2f}')
```

Predict 2

Quorvane's 100 is worth 76.61 today at 5.40 percent. The yield doubles to 10.80 percent. What is the 100 worth today?

A) near 38: the rate doubled, so the value halves

B) near 59: the value falls, by less than a quarter

C) near 75: the 100 has not changed, so the value barely moves

```
doubled = bondmath.present_value(100, 10.8, 5)
fall = (1 - doubled / bonds.loc[0, 'pv_of_100']) * 100
print(f'At 10.80 percent: {doubled:.4f}, down {fall:.1f} percent from 76.6118')
```

Predict 3

No bond here pays daily, but the formula does not care. 5.40 percent compounded 365 times a year: where does the 100 end after five years? Write down a number.

```
daily = 100 * (1 + 5.4 / 100 / 365) ** (365 * 5)
quarterly = bondmath.future_value(100, 5.4, 5, 4)
print(f'Daily: {daily:.4f}, only {daily - quarterly:.4f} above quarterly')
```

The note, answered

The note asked	Answer
100 grown at Quorvane's yield	130.5282, Excel's number
Each bond's 100, today	76.6118, 65.5527, 87.2461, 46.9753
The convention, in dollars	up to 65,052.95 on Halvering
Both functions, docstring first	bondmath.py
Match Excel on every row	3 passed, 36 rows and a round trip
A version to go back to	the first commit
Take it home	my_bondmath

You Can Now

Grow an amount with one formula: future value.

Bring a payment back to today: present value.

Spot a convention error in the frequency.

Write a function docstring first.

Test a module against Excel's answers with pytest, and read a pass and a failure.

Read Excel's "FV" and "PV" with their sign.

Make a first commit and take the files home.

Be proud: this is the first page of a module the desk will use for three weeks, and it already answers to Excel.

Next, Cash Flows and Price From Yield: a bond becomes a list of payments, and its price their sum.